Project Triply

Software requirements specification

TITLE PAGE

A. Project Title:
Collaborative Itinerary & Group Travel Planner

B. Project Adviser Name:
Dr. Amr Ghoniem

C. Project Team:
Shady Elaraby
Abdelrahman Mokhtar
Mohamed Atya
Nourhan Saeed
Mariam Elhadidy
Alaa Ahmed

D. Submission Date

E. Signatures of Project Adviser and Project Secondary Adviser

F. **Signatures of Team Members**

Table of Contents

Table of Contents iii
Revision History iv
1. Introduction 1
1.1 Review of Related Literature 1
1.2 Problem Statement 1
1.3 Proposed Solution 1
1.4 Problem Scope 1
1.5 Challenges 1
1.6 Knowledge Areas Required 1
1.7 Completeness Criteria 2
1.8 Research Outcomes/Nature of End Product 2
1.9 Learning Outcomes 2
1.10 Document Conventions 2
2. Background Study and Literature Survey 3
3. Overall Description 3
3.1 Proposed Solution 3
3.2 User Classes and Characteristics (if applicable) 3
3.3 Operating Environment 3
3.4 Design and Implementation Constraints 4
3.5 Assumptions and Dependencies 4
4. Functional Requirements 3
4.1 Requirement 1 OR Use-Case 1 (if requirements are modeled as use cases) 5
4.2 Requirement 2 OR Use-Case 2 (and so on) ..... **Error! Bookmark not defined.**
4.3 Proposed Workflow 1
4.4 Analysis and Modeling of Requirements 1
5. Nonfunctional Requirements 1
5.1 Target Performance ..... **Error! Bookmark not defined.**
5.2 Safety Requirements (if applicable) ..... **Error! Bookmark not defined.**
5.3 Security Requirements (if applicable) ..... **Error! Bookmark not defined.**
5.4 Additional Software Quality Attributes ..... **Error! Bookmark not defined.**
6. Other Requirements 5
7. Initial Results 5
8. Revised Project Plan 6
9. References 6
Appendix A: Glossary 6
Appendix B: IV & V Report 6
(Independent verification & validation) 6

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Review of Related Literature

Existing travel planning tools such as TripIt, Google Trips, and Splitwise each address isolated aspects of group travel — itinerary organisation, navigation, or expense splitting — but none consolidate these into a single collaborative environment. Research into group coordination software highlights a recurring gap: when travellers rely on multiple disconnected tools, communication overhead increases and decisions become fragmented.

Triply is positioned to close this gap by unifying itinerary management, expense tracking, group voting, and document storage under one platform.

1.2 Problem Statement

Planning a group trip is inherently a multi-person coordination problem. Groups currently juggle WhatsApp threads for decisions, Google Sheets for budgets, email chains for documents, and separate apps for itineraries. There is no single source of truth.

This leads to conflicting plans, missed contributions, unequal expense visibility, and general frustration particularly as group size grows.

1.3 Proposed Solution

Triply is a web-based collaborative trip planning platform that gives groups one shared workspace. A Trip Leader creates and manages a trip, invites members, and maintains the itinerary.

Members can RSVP to activities, log and split expenses, vote on group decisions via polls, upload travel documents, and receive automated notifications — all in real time and from any browser.

1.4 Problem Scope

Triply covers the full pre-trip and during-trip coordination lifecycle: itinerary building, activity scheduling, group expense management, democratic decision-making through polls, and travel document storage.

It does not handle flight or hotel booking, payment processing, or post-trip archiving beyond the trip's end date.

1.5 Challenges

The main technical challenges anticipated during development include implementing real-time synchronisation across multiple concurrent users, detecting and resolving scheduling conflicts in itineraries, handling multi-currency expense conversion accurately.

1.6 Knowledge Areas Required

- Web development (HTML, CSS, JavaScript)
- Backend development using PHP (Laravel)
- Database design (MySQL)
- Software Engineering concepts (SRS, UML, OOP)

1.7 Completeness Criteria

The system is considered complete when all core functionalities such as onboarding, session management, wellness tracking, and community interaction are implemented and tested successfully.

1.8 Research Outcomes/Nature of End Product

The end product is a functional, browser-based web application. The research component involves evaluating approaches to real-time collaborative state management in a PHP/MySQL environment.

1.9 Learning Outcomes

Through this project, students will gain experience in requirements engineering, system design, database development, and implementing secure web applications.

1.10 Document Conventions

This document follows a clear and structured format to ensure readability and consistency.
The following conventions are used throughout the document:

- Functional Requirements are labeled as **FR-XX** (e.g., FR-01, FR-02).
- Non-Functional Requirements are labeled as **NFR-XX** (e.g., NFR-SEC-01, NFR-02).
- Headings and subheadings are numbered hierarchically for easy navigation.
- Important terms may appear in **bold** for emphasis.
- Simple and clear language is used to ensure understanding by both technical and non-technical readers.
- Tables are used where appropriate to present structured information clearly.

2. Background Study and Literature Survey

Group travel coordination has been studied through the lens of both computer-supported cooperative work (CSCW) and consumer application design. Tools like TripIt automate itinerary parsing from email confirmations but offer no collaborative editing.

Splitwise handles expense fairness well but has no concept of a shared itinerary or group decision-making. Google Trips was discontinued in 2019, signalling that even large companies found the problem difficult to sustain commercially. Academic literature on shared planning systems consistently identifies three recurring failure modes: lack of a single authoritative data source, poor conflict resolution when multiple users edit simultaneously, and insufficient transparency around financial contributions.

Triply's design directly addresses each of these by introducing a server-authoritative sync model, a versioned itinerary with conflict detection, and a transparent shared expense ledger visible to all trip members..

3. Overall Description

3.1 Proposed Solution

Triply is built as a multi-tier web application. The frontend is rendered in HTML, CSS, and JavaScript and communicates with a PHP backend via RESTful HTTP requests. All persistent data — user accounts, trips, activities, expenses, polls, votes, and documents — is stored in a relational database (MySQL or PostgreSQL).

The system enforces a role hierarchy where a Trip Leader has elevated permissions over Members, and all access is controlled through token-based authentication. The architecture is modular, with each business domain (user management, itinerary, finance, collaboration, documents) maintained as a logically separate component.

3.2 User Classes and Characteristics (if applicable)

The system includes three main types of users:

1. Guest:

- an unauthenticated visitor.
- Views the landing page.
- Can register or log in.

2. Member: an authenticated user who has been invited to a trip. Can view the itinerary, RSVP to activities, log expenses, cast votes, upload documents, and invite others. Expected to have basic digital literacy.

3. Trip Leader:

a Member with elevated permissions. Can create and delete trips, confirm or remove activities, manage member permissions, and revert itinerary versions. Typically the person who initiated the trip.

3.3 Operating Environment

The system operates in a web-based environment and is accessible through modern browsers such as Chrome and Edge.

The backend is developed using PHP (Laravel), while the frontend uses HTML, CSS, JavaScript, and Bootstrap.

The system uses MySQL as the database for storing user and system data.

3.4 Design and Implementation Constraints

The system must follow the project requirements and constraints, including:

- Using PHP and Laravel for backend development
- Using MySQL as the database
- Developing a web-based system only
- Ensuring high levels of security and data protection
- Simulating advanced features such as therapist matching and crisis detection

3.5 Assumptions and Dependencies

The system is developed based on the following assumptions:

- Users have access to a stable internet connection
- Therapists provide valid and accurate credentials
- The system depends on the Laravel framework and MySQL database
- Some features are simulated and do not represent real clinical decisions
- External tools and APIs (if used) are compatible with the system

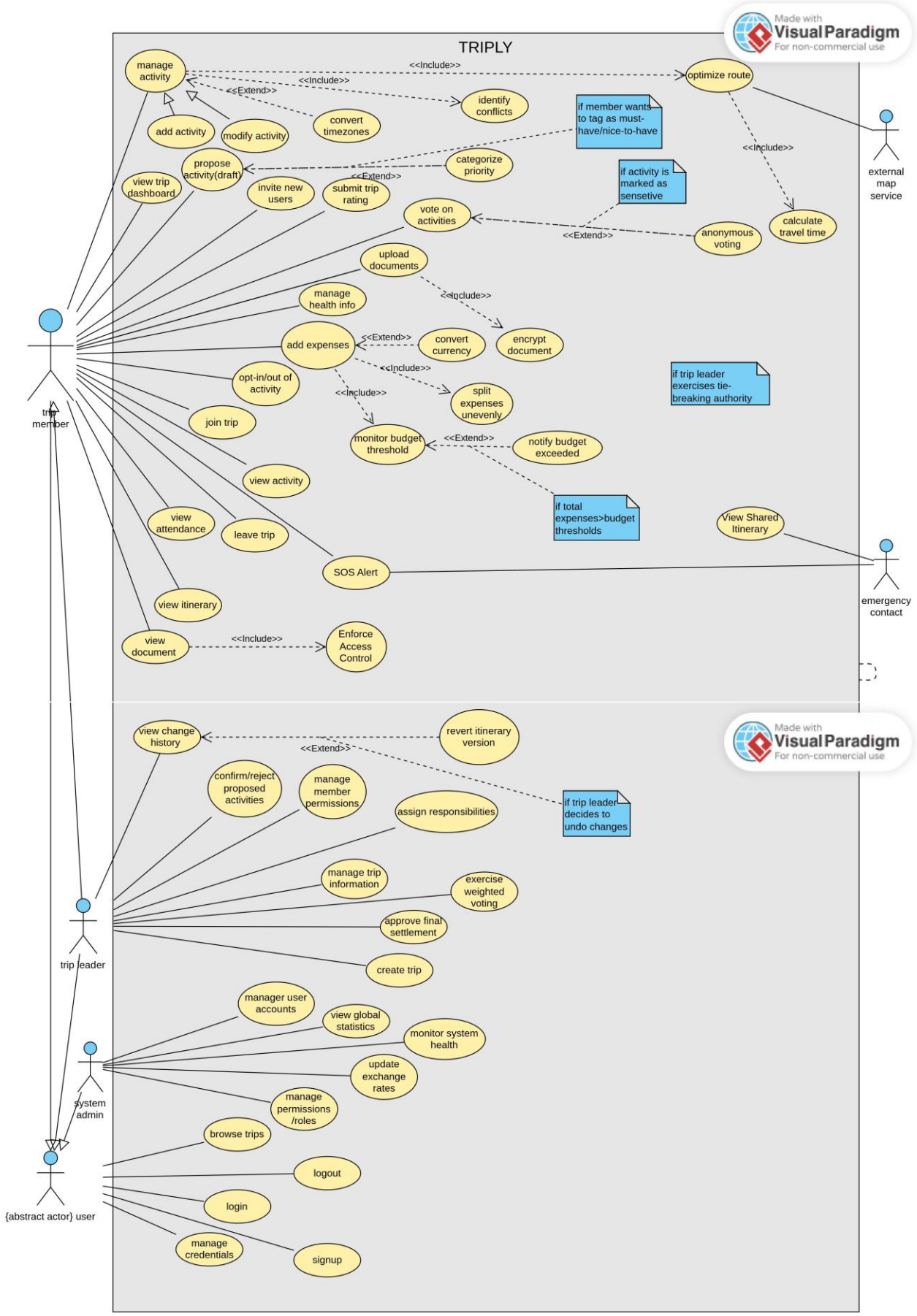
4. Functional Requirements

This section describes the system functional requirements of the system. Each requirement represents a core function that the system must perform to support users .

- **FR-01: The system shall identify and flag scheduling conflicts including overlapping activity and conflicting travel times between locations whenever a trip member adds or modifies an activity in the itinerary. [Must Have]**
- **FR-02: The system shall suggest and display the most logical order of activities in itinerary based on the physical proximity of their locations when a trip member requests route optimization. [Could Have]**
- **FR-03: The system shall record all changes made by members with their corresponding member's ID and timestamp allowing the group to view who added an activity and allow the trip leader to revert to any previously saved version. [Should Have]**
- **FR-04: The system shall automatically convert and display flight arrivals and activity schedules in the time zone of each place. [Should Have]**
- **FR-05: The system shall mark any activity proposed by a member as "Draft" by default, and allow the trip leader to confirm or reject it, changing its status to "Confirmed" or removing it. [Must Have]**
- **FR-06: The system shall ensure all members see itinerary changes reflected automatically. [Should Have]**
- **FR-07: The system shall automatically generate and send a daily notification to all trip members compiling the next day's schedule, weather and required tickets. [Could Have]**
- **FR-08: The system shall automatically insert a travel time between activities based on the estimated distance between their locations and mode of transport. [Should Have]**
- **FR-09: The system shall allow members to opt-in or out of specific activities and track the attendance count for each activity. [Must Have]**
- **FR-10: The system shall support handling expenses in multiple currencies and automatically convert all amounts into a predefined base trip currency using up-to-date exchange rates. [Must Have]**
- **FR-11: The system shall allow expenses to be split unevenly among members by specifying custom percentage or fixed amount contributions for each participant. [Must Have]**
- **FR-12: The system shall monitor the total trip expenses and notify all members when the predefined budget threshold is exceeded. [Must Have]**
- **FR-13: The system shall require all involved members to review and approve the final expense settlement before marking the trip as "Settled." [Should Have]**
- **FR-14: The system shall allow weighted voting where certain members (e.g., the trip organizer) have higher voting power or tie-breaking authority. [Should Have]**

- **FR-15: The system shall allow members to view a summary of all trip expenses, including the total amount spent and each member's individual share.**
- **FR-16: The system shall allow members to categorize activities as "Must-Have" or "Nice-to-Have" and prioritize them accordingly through voting. [Should Have]**
- **FR-17: The system shall automatically close voting polls when a predefined deadline is reached and select the option with the highest votes. [Should Have]**
- **FR-18: The system shall allow users to create and participate in activity-specific discussion threads linked to individual itinerary items. [Could Have]**
- **FR-19: The system shall allow anonymous voting to ensure privacy for sensitive decisions. [Should Have]**
- **FR-20: The system shall allow the trip organizer to assign or revoke editing permissions to other members. [Must Have]**
- **FR-21: The system shall provide a one-click mechanism to share the full group itinerary and member contact details with designated emergency contacts. [Should Have]**
- **FR-22: The system shall securely store uploaded documents (e.g., tickets, passport scans) using encryption mechanisms. [Must Have]**
- **FR-23: The system shall enforce access control policies ensuring only authorized users (e.g., owner and trip leader) can view sensitive documents. [Must Have]**
- **FR-24: The system shall check visa requirements based on each member's nationality and the destination country, and notify users if a visa is required. [Could Have]**
- **FR-25: The system shall allow members to assign and track responsibility for shared items (e.g., equipment) across the group. [Could Have]**
- **FR-26: The system shall maintain a secure repository of members' health information (e.g., allergies) accessible to the group when needed. [Should Have]**
- **FR-27: The system shall allow members to submit ratings and feedback after the trip is completed. [Could Have]**
- **FR-28: The system shall allow members to add proposed activities to a queue, and allow the trip leader to approve or reject each one before it is added to the itinerary. [Must Have]**
- **FR-29: The system shall allow existing members to invite new users to join a trip via invitation links or notifications. [Must Have]**
- **FR-30: The system shall award points to members when users they invited successfully join a trip. [Could Have]**

The following diagram illustrates the main use cases of the system.


TRIPLY
manage activity
optimize route
identify conflicts
convert timezones
add activity
modify activity
if member wants to tag as must-have/nice-to-have
propose activity(draft)
categorize priority
view trip dashboard
invite new users
submit trip rating
if activity is marked as sensetive
vote on activities
anonymous voting
calculate travel time
external map service
upload documents
manage health info
encrypt document
add expenses
convert currency
opt-in/out of activity
split expenses unevenly
if trip leader exercises tie-breaking authority
join trip
monitor budget threshold
notify budget exceeded
view activity
if total expenses>budget thresholds
view attendance
leave trip
View Shared Itinerary
SOS Alert
emergency contact
view itinerary
view document
Enforce Access Control
trip member
view change history
revert itinerary version
confirm/reject proposed activities
manage member permissions
assign responsibilities
if trip leader decides to undo changes
manage trip information
exercise weighted voting
approve final settlement
create trip
trip leader
manager user accounts
view global statistics
monitor system health
update exchange rates
manage permissions /roles
system admin
browse trips
logout
login
manage credentials
signup
{abstract actor} user
<<Include>>
<<Extend>>
Made with Visual Paradigm
For non-commercial use

UC-01

Trip Leader

Create Trip

Short-Description / Goal	Allows Trip Leader to create a new trip including destination, dates, and members
Preconditions	1. User is logged in 2. User is Trip Leader
Postconditions	Trip is created with initial details and stored in system
Error Situations - System State in the Event of an Error	1. Missing required fields → trip not created 2. Invalid dates → system rejects input
Actor(s) / Initiator(s)	Trip Leader
Trigger	User clicks 'Create Trip'
Standard Process (Main Success Scenario)	1. User opens create trip page 2. User enters destination, dates, and description 3. User optionally adds members 4. User submits form 5. System validates data 6. System stores trip information 7. System confirms creation
Alternative Processes (Alternative/Other Scenarios)	Invalid data → system shows error and asks for correction

UC-02

Trip Leader

Delete Trip

Short-Description / Goal	Allows Trip Leader to permanently delete an existing trip
Preconditions	1. User is logged in 2. User is Trip Leader 3. Trip exists
Postconditions	Trip and all associated data are removed from the system
Error Situations - System State in the Event of an Error	1. Trip not found → error shown 2. User lacks permission → action denied
Actor(s) / Initiator(s)	Trip Leader
Trigger	User clicks 'Delete Trip'
Standard Process (Main Success Scenario)	1. User opens trip settings 2. User clicks delete trip 3. System asks for confirmation 4. User confirms deletion 5. System deletes trip and all related data 6. System redirects to home page
Alternative Processes (Alternative/Other Scenarios)	User cancels → trip is not deleted

UC-03

Trip Leader

Update Trip

Short-Description / Goal	Allows Trip Leader to update trip information such as dates or destination
Preconditions	1. User is logged in 2. User is Trip Leader 3. Trip exists
Postconditions	Trip details are updated and saved
Error Situations - System State in the Event of an Error	1. Invalid data → system rejects and shows error 2. Network error → changes not saved
Actor(s) / Initiator(s)	Trip Leader
Trigger	User edits trip details
Standard Process (Main Success Scenario)	1. User opens trip settings 2. User edits desired fields 3. User submits changes 4. System validates updated data 5. System saves changes 6. System confirms update
Alternative Processes (Alternative/Other Scenarios)	Invalid input → system highlights errors

UC-04

Trip Member

Add/Modify Activity

Short-Description / Goal	Allows a trip member to add a new activity or modify an existing one in the trip itinerary
Preconditions	1. User is logged in 2. User is a member of the trip
Postconditions	Activity is added or updated in the trip itinerary
Error Situations - System State in the Event of an Error	1. Missing required fields → activity not saved 2. Scheduling conflict detected → system notifies user
Actor(s) / Initiator(s)	Trip Member
Trigger	User selects 'Add Activity' or edits existing one
Standard Process (Main Success Scenario)	1. User opens trip itinerary 2. User clicks 'Add Activity' or selects existing activity 3. User fills in activity details 4. System checks for scheduling conflicts 5. System identifies conflicting timezones if needed 6. User submits 7. System saves activity
Alternative Processes (Alternative/Other Scenarios)	Conflict detected → user is prompted to resolve

UC-05

Trip Member

Propose Activity (Draft)

Short-Description / Goal	Allows a trip member to propose an activity as a draft for group consideration
Preconditions	1. User is logged in 2. User is a member of the trip
Postconditions	Draft activity is submitted and visible to other members
Error Situations - System State in the Event of an Error	1. Incomplete proposal → draft not submitted
Actor(s) / Initiator(s)	Trip Member
Trigger	User submits a draft proposal
Standard Process (Main Success Scenario)	1. User opens proposal form 2. User fills activity details as draft 3. User submits draft 4. System saves draft and notifies members
Alternative Processes (Alternative/Other Scenarios)	User saves without submitting → stays as local draft

UC-06

Trip Member

Opt-in/Out of Activity

Short-Description / Goal	Allows a trip member to opt in or out of a specific activity
Preconditions	1. User is logged in 2. Activity exists in the itinerary
Postconditions	User's participation status is updated
Error Situations - System State in the Event of an Error	1. Activity already started → opt-out not allowed
Actor(s) / Initiator(s)	Trip Member
Trigger	User toggles participation in an activity
Standard Process (Main Success Scenario)	1. User views activity 2. User clicks 'Join' or 'Leave' 3. System updates participation list 4. System notifies trip leader if needed
Alternative Processes (Alternative/Other Scenarios)	Activity is mandatory → opt-out is disabled

UC-07

Trip Member

Add Expenses

Short-Description / Goal	Allows trip members to log expenses related to the trip
Preconditions	1. User is logged in 2. User is a member of the trip
Postconditions	Expense is recorded; budget is updated; currency converted if needed
Error Situations - System State in the Event of an Error	1. Invalid amount → expense not saved 2. Budget exceeded → notification sent
Actor(s) / Initiator(s)	Trip Member
Trigger	User clicks 'Add Expense'
Standard Process (Main Success Scenario)	1. User opens expenses section 2. User enters expense details and amount 3. System converts currency if needed 4. System saves expense 5. System updates trip budget 6. System monitors budget threshold 7. System notifies if budget exceeded
Alternative Processes (Alternative/Other Scenarios)	Invalid input → error shown, expense not recorded

UC-08

Trip Member

Vote on Activities

Short-Description / Goal	Allows trip members to vote on proposed activities
Preconditions	1. User is logged in 2. User is a member of the trip 3. Voting is active for an activity
Postconditions	Vote is recorded and tally updated
Error Situations - System State in the Event of an Error	1. Voting period has ended → vote not accepted 2. User already voted → duplicate prevented
Actor(s) / Initiator(s)	Trip Member
Trigger	Voting is opened for an activity
Standard Process (Main Success Scenario)	1. User opens voting section 2. User views activity details 3. User casts vote 4. System records vote 5. System updates vote tally 6. Voting auto-closes when deadline reached
Alternative Processes (Alternative/Other Scenarios)	Activity is sensitive → anonymous voting is triggered

UC-09

Trip Member

Participate in Discussions

Short-Description / Goal	Allows trip members to communicate and discuss trip plans
Preconditions	1. User is logged in 2. User is a member of the trip
Postconditions	Message is posted and visible to all members
Error Situations - System State in the Event of an Error	1. Empty message → not posted 2. User muted → cannot post
Actor(s) / Initiator(s)	Trip Member
Trigger	User opens discussion board
Standard Process (Main Success Scenario)	1. User opens discussion section 2. User types message 3. User submits message 4. System validates and posts message 5. Other members receive notification
Alternative Processes (Alternative/Other Scenarios)	User edits or deletes message → system updates thread

UC-10

Trip Member

Submit Trip Rating

Short-Description / Goal	Allows members to rate the overall trip experience
Preconditions	1. User is logged in 2. Trip is completed
Postconditions	Rating is submitted and stored
Error Situations - System State in the Event of an Error	1. Rating already submitted → duplicate prevented
Actor(s) / Initiator(s)	Trip Member
Trigger	Trip is marked as completed
Standard Process (Main Success Scenario)	1. System prompts user to rate trip 2. User selects rating score 3. User optionally adds comment 4. System saves rating 5. System updates trip rating average
Alternative Processes (Alternative/Other Scenarios)	User skips → no rating submitted

UC-11

Trip Member

Approve Final Settlement

Short-Description / Goal	Allows members to review and approve the final expense settlement
Preconditions	1. User is logged in 2. Trip has ended 3. All expenses are logged
Postconditions	Settlement is approved and finalized
Error Situations - System State in the Event of an Error	1. Member disputes amount → settlement flagged for review
Actor(s) / Initiator(s)	Trip Member
Trigger	Settlement is presented at end of trip
Standard Process (Main Success Scenario)	1. System calculates final settlement 2. System presents each member's share 3. User reviews settlement 4. User approves or disputes 5. If approved by all → settlement finalized 6. System sends confirmation to all members
Alternative Processes (Alternative/Other Scenarios)	Member disputes → admin or leader reviews

UC-12

System (included in Add/Modify Activity)

Identify Conflicts

Short-Description / Goal	System automatically detects scheduling conflicts between activities
Preconditions	1. At least two activities exist in the itinerary
Postconditions	Conflicts are identified and user is notified
Error Situations - System State in the Event of an Error	1. System fails to retrieve schedule → conflict check skipped
Actor(s) / Initiator(s)	System (included in Add/Modify Activity)
Trigger	Triggered automatically when activity is added or modified
Standard Process (Main Success Scenario)	1. User adds or modifies activity 2. System compares new activity time with existing ones 3. System detects any overlap 4. System notifies user of conflict 5. User resolves conflict manually
Alternative Processes (Alternative/Other Scenarios)	No conflict found → activity saved normally

UC-13

System (included in Add/Modify Activity)

Convert Timezones

Short-Description / Goal	System converts activity times to correct local timezones for all members
Preconditions	1. Activity has a time and destination set 2. Members are in different timezones
Postconditions	Activity times are displayed in each member's local timezone
Error Situations - System State in the Event of an Error	1. Timezone data unavailable → UTC displayed as fallback
Actor(s) / Initiator(s)	System (included in Add/Modify Activity)
Trigger	Triggered when activity spans multiple timezones
Standard Process (Main Success Scenario)	1. System detects destination timezone 2. System retrieves each member's local timezone 3. System converts activity time accordingly 4. System displays localized times to each member
Alternative Processes (Alternative/Other Scenarios)	Member sets manual timezone → system uses that instead

UC-14

Trip Member

Categorize Priority

Short-Description / Goal	Allows members to tag activities with a priority level to aid planning
Preconditions	1. User is logged in 2. Activity exists in itinerary
Postconditions	Activity is tagged with selected priority level
Error Situations - System State in the Event of an Error	1. Activity not found → tagging fails
Actor(s) / Initiator(s)	Trip Member
Trigger	Member tags activity as must-have or nice-to-have
Standard Process (Main Success Scenario)	1. User opens activity 2. User selects priority tag (must-have / nice-to-have) 3. System saves tag 4. System updates activity display with priority badge
Alternative Processes (Alternative/Other Scenarios)	User removes tag → activity returns to untagged state

UC-15

Trip Member

Invite New Users

Short-Description / Goal	Allows trip members to invite new users to join the trip
Preconditions	1. User is logged in 2. User is a member of the trip
Postconditions	Invitation is sent to the new user
Error Situations - System State in the Event of an Error	1. Invalid email → invitation not sent 2. User already in trip → duplicate prevented
Actor(s) / Initiator(s)	Trip Member
Trigger	User clicks 'Invite'
Standard Process (Main Success Scenario)	1. User opens invite section 2. User enters email of invitee 3. System validates email 4. System sends invitation 5. Invitee receives email and joins
Alternative Processes (Alternative/Other Scenarios)	Invitee ignores → invitation expires after set time

UC-16

System (included in Upload Documents)

Encrypt Document

Short-Description / Goal	System encrypts uploaded documents to ensure secure storage
Preconditions	1. Document has been uploaded by user
Postconditions	Document is encrypted and stored securely
Error Situations - System State in the Event of an Error	1. Encryption fails → document not stored, user notified
Actor(s) / Initiator(s)	System (included in Upload Documents)
Trigger	Triggered automatically when document is uploaded
Standard Process (Main Success Scenario)	1. User uploads document 2. System receives file 3. System encrypts document using secure algorithm 4. System stores encrypted file 5. System logs encryption event
Alternative Processes (Alternative/Other Scenarios)	Encryption service unavailable → upload aborted

UC-17

System (extends Monitor Budget Threshold)

Notify Budget Exceeded

Short-Description / Goal	System automatically notifies members when the trip budget is exceeded
Preconditions	1. Budget threshold is set 2. Expenses have been added
Postconditions	Notification is sent to all relevant members
Error Situations - System State in the Event of an Error	1. Notification service down → alert logged but not sent
Actor(s) / Initiator(s)	System (extends Monitor Budget Threshold)
Trigger	Total expenses exceed the defined budget threshold
Standard Process (Main Success Scenario)	1. System monitors total expenses after each addition 2. System compares total to budget threshold 3. If threshold exceeded → system triggers alert 4. System sends notification to trip leader and members
Alternative Processes (Alternative/Other Scenarios)	Budget updated to higher limit → alert cleared

UC-18

System (extends Vote on Activities)

Anonymous Voting

Short-Description / Goal	Ensures that votes on sensitive activities are cast anonymously
Preconditions	1. Activity is marked as sensitive 2. Voting is active
Postconditions	Votes are recorded without revealing voter identity
Error Situations - System State in the Event of an Error	1. Anonymization fails → voting paused
Actor(s) / Initiator(s)	System (extends Vote on Activities)
Trigger	Activity is marked as sensitive
Standard Process (Main Success Scenario)	1. System detects activity is marked sensitive 2. System enables anonymous mode for voting 3. Members cast votes without names attached 4. System tallies votes 5. System reveals result only, not individual votes
Alternative Processes (Alternative/Other Scenarios)	Activity unmarked as sensitive → normal voting resumes

UC-19

Trip Member / Trip Leader

Exercise Weighted Voting

Short-Description / Goal	Allows certain members to have more voting weight based on role or setting
Preconditions	1. Weighted voting is enabled 2. Voting is active
Postconditions	Votes are tallied using assigned weights
Error Situations - System State in the Event of an Error	1. Weight values invalid → system uses default equal weights
Actor(s) / Initiator(s)	Trip Member / Trip Leader
Trigger	Trip leader enables weighted voting
Standard Process (Main Success Scenario)	1. Trip leader enables weighted voting 2. System assigns weight values to members 3. Members cast votes 4. System multiplies votes by assigned weights 5. System tallies weighted results 6. System displays outcome
Alternative Processes (Alternative/Other Scenarios)	Tie occurs → trip leader exercises tiebreaking authority

UC-20

System (extends Vote on Activities)

Auto Close Voting

Short-Description / Goal	System automatically closes voting when the deadline is reached
Preconditions	1. Voting is active 2. A deadline has been set
Postconditions	Voting is closed and final tally is recorded
Error Situations - System State in the Event of an Error	1. System clock error → voting remains open until manual close
Actor(s) / Initiator(s)	System (extends Vote on Activities)
Trigger	Voting deadline is reached
Standard Process (Main Success Scenario)	1. System monitors voting deadline 2. Deadline is reached 3. System closes voting automatically 4. System records final tally 5. System notifies all members of result
Alternative Processes (Alternative/Other Scenarios)	All members vote early → voting closes before deadline

UC-21

Trip Member

Upload Documents

Short-Description / Goal	Allows trip members to upload trip-related documents such as visas and tickets
Preconditions	1. User is logged in 2. User is a member of the trip
Postconditions	Document is uploaded, encrypted, and accessible to authorized members
Error Situations - System State in the Event of an Error	1. Unsupported file type → upload rejected 2. File too large → upload rejected
Actor(s) / Initiator(s)	Trip Member
Trigger	User clicks 'Upload Document'
Standard Process (Main Success Scenario)	1. User opens documents section 2. User selects file to upload 3. System validates file 4. System encrypts document 5. System stores document 6. System grants access based on permissions
Alternative Processes (Alternative/Other Scenarios)	Encryption fails → upload aborted

UC-22

System (included in Approve Final Settlement)

Split Expenses Unevenly

Short-Description / Goal	System supports splitting expenses unevenly among trip members
Preconditions	1. Expenses have been logged 2. Members agree to custom split
Postconditions	Expenses are split according to custom percentages or amounts
Error Situations - System State in the Event of an Error	1. Split percentages do not total 100% → error shown
Actor(s) / Initiator(s)	System (included in Approve Final Settlement)
Trigger	Members agree to uneven expense split
Standard Process (Main Success Scenario)	1. User opens settlement settings 2. User selects uneven split option 3. User enters custom share for each member 4. System validates total 5. System applies custom split 6. System displays updated shares
Alternative Processes (Alternative/Other Scenarios)	Member rejects split → negotiation continues

UC-23

System (included in Add Expenses)

Convert Currency

Short-Description / Goal	System automatically converts foreign currency expenses to the trip's base currency
Preconditions	1. Exchange rates are up to date 2. Expense has a currency specified
Postconditions	Expense is converted and stored in base currency
Error Situations - System State in the Event of an Error	1. Exchange rate unavailable → user prompted to enter manually
Actor(s) / Initiator(s)	System (included in Add Expenses)
Trigger	Expense is entered in a foreign currency
Standard Process (Main Success Scenario)	1. User enters expense in foreign currency 2. System detects currency mismatch 3. System fetches current exchange rate 4. System converts amount 5. System stores converted expense
Alternative Processes (Alternative/Other Scenarios)	Rate outdated → system flags and prompts admin to update

UC-24

System (included in Add Expenses)

Monitor Budget Threshold

Short-Description / Goal	System continuously monitors total expenses against the defined budget threshold
Preconditions	1. Budget is set for the trip 2. Expenses are being added
Postconditions	Budget status is updated; alert triggered if threshold exceeded
Error Situations - System State in the Event of an Error	1. Budget not set → monitoring skipped
Actor(s) / Initiator(s)	System (included in Add Expenses)
Trigger	Expense is added to the trip
Standard Process (Main Success Scenario)	1. System retrieves current total expenses 2. System compares total to budget threshold 3. If below threshold → no action 4. If threshold reached or exceeded → notify budget exceeded use case triggered
Alternative Processes (Alternative/Other Scenarios)	Budget threshold changed → system recalculates immediately

UC-25

Trip Member

Manage Health Info

Short-Description / Goal	Allows members to manage their sensitive health information for trip planning
Preconditions	1. User is logged in
Postconditions	Health info is saved securely
Error Situations - System State in the Event of an Error	1. Incomplete form → not saved
Actor(s) / Initiator(s)	Trip Member
Trigger	User opens health info settings
Standard Process (Main Success Scenario)	1. User opens health settings 2. User enters or updates health details 3. System saves data securely 4. System marks data as sensitive
Alternative Processes (Alternative/Other Scenarios)	User cancels → no changes saved

UC-26

Trip Member

View Trip

Short-Description / Goal	Allows members to view all trip details including itinerary and members
Preconditions	1. User is logged in 2. User is a member of the trip
Postconditions	Trip details are displayed
Error Situations - System State in the Event of an Error	1. Trip not found → error shown
Actor(s) / Initiator(s)	Trip Member
Trigger	User opens trip page
Standard Process (Main Success Scenario)	1. User clicks on trip 2. System retrieves trip data 3. System displays itinerary, members, activities, and expenses
Alternative Processes (Alternative/Other Scenarios)	Trip is private → access denied for non-members

UC-27

Trip Member / Trip Leader

Assign Responsibilities

Short-Description / Goal	Allows leaders or members to assign specific tasks to trip members
Preconditions	1. User is logged in 2. Trip exists
Postconditions	Responsibility is assigned and member is notified
Error Situations - System State in the Event of an Error	1. Member not found → assignment fails
Actor(s) / Initiator(s)	Trip Member / Trip Leader
Trigger	User opens responsibilities section
Standard Process (Main Success Scenario)	1. User opens responsibilities section 2. User selects task 3. User assigns task to a member 4. System saves assignment 5. System notifies assigned member
Alternative Processes (Alternative/Other Scenarios)	Member declines → task returned to unassigned

UC-28

External System

Optimize Route

Short-Description / Goal	External system provides optimized travel routes for the trip itinerary
Preconditions	1. Trip has multiple destinations
Postconditions	Optimized route is returned and displayed
Error Situations - System State in the Event of an Error	1. External system unavailable → fallback message shown
Actor(s) / Initiator(s)	External System
Trigger	System requests route optimization
Standard Process (Main Success Scenario)	1. System sends destinations to external API 2. External system calculates optimal route 3. System receives route data 4. System displays route on map
Alternative Processes (Alternative/Other Scenarios)	No optimal route → multiple options presented

UC-29

External System

Check Visa Requirements

Short-Description / Goal	External system provides visa requirement info based on nationality and destination
Preconditions	1. User is logged in 2. Destination is set
Postconditions	Visa requirements are displayed
Error Situations - System State in the Event of an Error	1. External API error → cached data shown or error message
Actor(s) / Initiator(s)	External System
Trigger	User requests visa info for destination
Standard Process (Main Success Scenario)	1. User requests visa info 2. System sends request to external API 3. API returns visa requirements 4. System displays results to user
Alternative Processes (Alternative/Other Scenarios)	Country not found → user prompted to check manually

UC-30

External System

Recommend Activity

Short-Description / Goal	External system recommends activities based on destination and user preferences
Preconditions	1. Trip has a destination set
Postconditions	Activity recommendations are displayed
Error Situations - System State in the Event of an Error	1. API unavailable → no recommendations shown
Actor(s) / Initiator(s)	External System
Trigger	System requests activity suggestions
Standard Process (Main Success Scenario)	1. System sends destination to external recommendation API 2. API returns suggested activities 3. System displays recommendations to members
Alternative Processes (Alternative/Other Scenarios)	No recommendations → system shows generic popular activities

UC-31

System (included in Add/Modify Activity)

Calculate Travel Time

Short-Description / Goal	System calculates estimated travel time between activity locations
Preconditions	1. At least two activity locations exist 2. External mapping service is available
Postconditions	Travel time is calculated and displayed between activities
Error Situations - System State in the Event of an Error	1. Mapping service unavailable → travel time not shown
Actor(s) / Initiator(s)	System (included in Add/Modify Activity)
Trigger	Activity with travel is added to itinerary
Standard Process (Main Success Scenario)	1. User adds activity with a location 2. System retrieves previous activity location 3. System requests travel time from mapping service 4. System receives estimated duration 5. System displays travel time between activities
Alternative Processes (Alternative/Other Scenarios)	No route found → user notified to verify manually

UC-32

Trip Leader

Confirm/Reject Proposed Activities

Short-Description / Goal	Allows Trip Leader to approve or reject activities proposed by members
Preconditions	1. User is logged in 2. User is Trip Leader 3. At least one activity proposal exists
Postconditions	Activity is confirmed or rejected; members are notified
Error Situations - System State in the Event of an Error	1. No proposals exist → nothing to review
Actor(s) / Initiator(s)	Trip Leader
Trigger	Trip leader reviews proposed activities
Standard Process (Main Success Scenario)	1. Trip Leader opens proposals list 2. Trip Leader reviews each proposed activity 3. Trip Leader clicks confirm or reject 4. System updates activity status 5. System notifies relevant members
Alternative Processes (Alternative/Other Scenarios)	Leader leaves without action → proposals remain pending

UC-33

Trip Leader

Revert Itinerary Version

Short-Description / Goal	Allows the trip leader to revert the trip itinerary to a previous version
Preconditions	1. User is logged in 2. User is Trip Leader 3. Change history exists
Postconditions	Itinerary is reverted to the selected previous version
Error Situations - System State in the Event of an Error	1. No previous version available → revert not possible
Actor(s) / Initiator(s)	Trip Leader
Trigger	Trip leader decides to undo changes
Standard Process (Main Success Scenario)	1. Trip Leader opens change history 2. Trip Leader selects a previous version 3. Trip Leader clicks revert 4. System asks for confirmation 5. System restores selected version 6. System notifies all members of the revert
Alternative Processes (Alternative/Other Scenarios)	Leader cancels → current version remains unchanged

UC-34

Trip Leader

Manage Member Permissions

Short-Description / Goal	Allows trip leader to control what actions each member can perform
Preconditions	1. User is logged in 2. User is Trip Leader
Postconditions	Permissions are updated and applied
Error Situations - System State in the Event of an Error	1. Invalid permission level → not applied
Actor(s) / Initiator(s)	Trip Leader
Trigger	User opens member settings
Standard Process (Main Success Scenario)	1. User opens member management 2. User selects member 3. User assigns or removes permissions 4. System saves changes 5. System applies new access rules
Alternative Processes (Alternative/Other Scenarios)	Leader removes self → system prevents action

UC-35

Trip Leader

View Change History

Short-Description / Goal	Allows the trip leader to view a log of all changes made to the trip
Preconditions	1. User is logged in 2. User is Trip Leader
Postconditions	Change history is displayed
Error Situations - System State in the Event of an Error	1. No changes recorded → empty log shown
Actor(s) / Initiator(s)	Trip Leader
Trigger	User opens change history log
Standard Process (Main Success Scenario)	1. User opens change history 2. System retrieves log 3. System displays list of changes with timestamps and authors
Alternative Processes (Alternative/Other Scenarios)	Leader reverts change → itinerary rolled back

UC-36

Trip Leader

Access Control for Doc

Short-Description / Goal	Allows the trip leader to control who can view or edit uploaded documents
Preconditions	1. User is logged in 2. User is Trip Leader 3. Documents exist in the trip
Postconditions	Access permissions are applied to the selected document
Error Situations - System State in the Event of an Error	1. Permission conflict → system applies most restrictive rule
Actor(s) / Initiator(s)	Trip Leader
Trigger	Leader sets document access permissions
Standard Process (Main Success Scenario)	1. Trip Leader opens documents section 2. Trip Leader selects a document 3. Trip Leader sets access permissions per member 4. System saves and applies permissions 5. Members see only documents they are authorized to view
Alternative Processes (Alternative/Other Scenarios)	Leader grants full access → all members can view and edit

UC-37

Emergency Contact

Receive Trip Itinerary

Short-Description / Goal	Allows an emergency contact to receive a copy of the trip itinerary
Preconditions	1. Trip itinerary is finalized 2. Emergency contact info is provided
Postconditions	Emergency contact receives the trip itinerary
Error Situations - System State in the Event of an Error	1. Invalid contact info → itinerary not delivered
Actor(s) / Initiator(s)	Emergency Contact
Trigger	Trip leader shares itinerary with emergency contact
Standard Process (Main Success Scenario)	1. Trip Leader triggers share to emergency contact 2. System retrieves finalized itinerary 3. System sends itinerary to emergency contact via email 4. Emergency contact receives itinerary
Alternative Processes (Alternative/Other Scenarios)	Contact unreachable → system retries or notifies leader

UC-38

Trip Member

Share Itinerary Member

Short-Description / Goal	Allows members to share the trip itinerary with an emergency contact
Preconditions	1. User is logged in 2. Trip itinerary exists
Postconditions	Itinerary is shared with the emergency contact
Error Situations - System State in the Event of an Error	1. Sharing fails → member notified
Actor(s) / Initiator(s)	Trip Member
Trigger	Member shares itinerary with emergency contact
Standard Process (Main Success Scenario)	1. User opens itinerary 2. User clicks share 3. User enters emergency contact details 4. System sends itinerary copy 5. System confirms delivery
Alternative Processes (Alternative/Other Scenarios)	Contact email invalid → user prompted to re-enter

UC-39

Emergency Contact

Receive Emergency Alert

Short-Description / Goal	Emergency contact receives an alert when an emergency is reported during the trip
Preconditions	1. Emergency contact info is registered 2. An emergency is reported by a member
Postconditions	Emergency contact is notified with trip and member details
Error Situations - System State in the Event of an Error	1. Contact unreachable → alert logged and retried
Actor(s) / Initiator(s)	Emergency Contact
Trigger	Emergency alert is triggered during the trip
Standard Process (Main Success Scenario)	1. Member or system triggers emergency alert 2. System retrieves emergency contact info 3. System sends alert with member and trip details 4. Emergency contact receives notification 5. System logs alert event
Alternative Processes (Alternative/Other Scenarios)	No emergency contact set → alert sent to system admin

UC-40

User (abstract)

Login

Short-Description / Goal	Allows any user to authenticate and access the system
Preconditions	1. User has an account
Postconditions	User is authenticated and redirected to dashboard
Error Situations - System State in the Event of an Error	1. Wrong credentials → access denied 2. Account locked → user notified
Actor(s) / Initiator(s)	User (abstract)
Trigger	User opens login page
Standard Process (Main Success Scenario)	1. User opens login page 2. User enters email and password 3. System validates credentials 4. System grants access 5. System redirects to dashboard
Alternative Processes (Alternative/Other Scenarios)	Forgot password → user redirected to reset flow

UC-41

User (abstract)

Logout

Short-Description / Goal	Allows user to securely end their session
Preconditions	1. User is logged in
Postconditions	Session is terminated
Error Situations - System State in the Event of an Error	1. Session already expired → handled gracefully
Actor(s) / Initiator(s)	User (abstract)
Trigger	User clicks 'Logout'
Standard Process (Main Success Scenario)	1. User clicks logout 2. System ends user session 3. System redirects to login page
Alternative Processes (Alternative/Other Scenarios)	Session timeout → auto logout

UC-42

User (abstract)

Manage Profile

Short-Description / Goal	Allows users to view and edit their personal profile information
Preconditions	1. User is logged in
Postconditions	Profile is updated
Error Situations - System State in the Event of an Error	1. Invalid data → not saved
Actor(s) / Initiator(s)	User (abstract)
Trigger	User opens profile settings
Standard Process (Main Success Scenario)	1. User opens profile page 2. User edits personal info 3. User saves changes 4. System validates and saves
Alternative Processes (Alternative/Other Scenarios)	User cancels → no changes applied

UC-43

User (abstract)

Change Password

Short-Description / Goal	Allows users to update their account password
Preconditions	1. User is logged in
Postconditions	Password is changed successfully
Error Situations - System State in the Event of an Error	1. Old password incorrect → rejected 2. New password too weak → rejected
Actor(s) / Initiator(s)	User (abstract)
Trigger	User requests password change
Standard Process (Main Success Scenario)	1. User opens security settings 2. User enters old password 3. User enters new password 4. System validates 5. System updates password 6. System confirms change
Alternative Processes (Alternative/Other Scenarios)	Forgot password → email reset link sent

UC-44

System Admin

Manager User Accounts

Short-Description / Goal	Allows system admin to create, edit, suspend, or delete user accounts
Preconditions	1. Admin is logged in
Postconditions	User account changes are saved
Error Situations - System State in the Event of an Error	1. User not found → error shown 2. Cannot delete active admin → prevented
Actor(s) / Initiator(s)	System Admin
Trigger	Admin opens user management panel
Standard Process (Main Success Scenario)	1. Admin opens user management 2. Admin searches for user 3. Admin selects action (edit/suspend/delete) 4. System performs action 5. System logs the operation
Alternative Processes (Alternative/Other Scenarios)	Invalid action → error message shown

UC-45

System Admin

Monitor System Health

Short-Description / Goal	Allows admin to monitor server status, uptime, and performance metrics
Preconditions	1. Admin is logged in
Postconditions	System health status is displayed
Error Situations - System State in the Event of an Error	1. Monitoring service down → fallback message shown
Actor(s) / Initiator(s)	System Admin
Trigger	Admin opens system dashboard
Standard Process (Main Success Scenario)	1. Admin opens system health panel 2. System retrieves health metrics 3. System displays uptime, load, and alerts
Alternative Processes (Alternative/Other Scenarios)	Alert triggered → admin receives notification

UC-46

System Admin

Update Exchange Rates

Short-Description / Goal	Allows admin to update currency exchange rates used across the system
Preconditions	1. Admin is logged in
Postconditions	Exchange rates are updated system-wide
Error Situations - System State in the Event of an Error	1. Invalid rate data → not applied
Actor(s) / Initiator(s)	System Admin
Trigger	Admin triggers rate update
Standard Process (Main Success Scenario)	1. Admin opens exchange rate settings 2. Admin inputs or fetches new rates 3. System validates rates 4. System saves and applies rates
Alternative Processes (Alternative/Other Scenarios)	Fetch fails → admin enters rates manually

UC-47

System Admin

View Global Statistics

Short-Description / Goal	Allows admin to view aggregated system usage and trip statistics
Preconditions	1. Admin is logged in
Postconditions	Statistics are displayed
Error Situations - System State in the Event of an Error	1. No data available → empty state shown
Actor(s) / Initiator(s)	System Admin
Trigger	Admin opens statistics panel
Standard Process (Main Success Scenario)	1. Admin opens statistics panel 2. System aggregates data 3. System displays charts and metrics
Alternative Processes (Alternative/Other Scenarios)	Filter applied → data refreshed by criteria

4.1 Proposed Workflow

The typical workflow of the system starts when a patient registers and logs into the platform. The patient then fills out the intake form, which is analyzed by the system to match them with a suitable therapist.

After selecting a therapist, the patient schedules a session based on available time slots. The system sends reminders before the session time.

During the scheduled time, the patient joins the session with the therapist. After the session, the patient can track their mood, set wellness goals, and access self-help resources.

Additionally, the patient can participate in anonymous community discussions. Admins monitor the system and moderate content to ensure safety and compliance.

4.2 Analysis and Modeling of Requirements

The system requirements are analyzed and modeled using appropriate software engineering diagrams. The following models are used:

- Use Case Diagram: to represent the interaction between users and the system
- Activity Diagrams: to illustrate the workflow of key system processes
- Class Diagram: to represent the system structure and object relationships
- Object Diagram: to provide examples of system instances at runtime

These models help in understanding the system behavior, structure, and interactions between components.
Additionally, the system follows a layered architecture consisting of presentation, business logic, and data layers to ensure modularity and scalability.

5. Nonfunctional Requirements

3. Non-Functional Requirements (NFRs)

This section specifies the quality attributes and operational constraints that the Collaborative Group Travel Planner system must satisfy. All requirements are stated in measurable, testable terms and are independent of any specific implementation decisions unless otherwise noted. Requirements are classified using the MoSCoW scheme and are traceable to the system architecture described in Section 2.

6. 3.0 NFR Summary Table

Table 3.0 provides a consolidated overview of all non-functional requirements. Detailed specifications follow in Sections 3.1 through 3.5.

Category	Requirement Description	Metric / Measurement	Priority
Performance	Itinerary rendering for up to 20 activities	≤ 3.0 seconds	**High**
Performance	HTTP response time under concurrent load (100 users)	≤ 500 ms	**High**
Performance	In-memory retrieval of cached session/draft data	< 50 ms	**High**
Security	Encryption of sensitive documents at rest	AES-256 or equivalent	**High**
Security	Token-based API authentication with expiry	Access: 1 hr / Refresh: 24 hr	**High**
Security	RBAC enforcement on financial write-access	0 unauthorised write-accesses	**High**
Security	Rate limiting on public endpoints	100 req/min per user	**High**
Reliability	Offline access to confirmed itineraries	100% availability offline	**High**
Reliability	State synchronisation upon connectivity restore	≤ 5 seconds	**High**
Reliability	Automated daily data backup with retention	7-day retention	**Medium**
Usability	Task completion within minimal interaction steps	3–5 clicks/taps	**Medium**
Usability	Graceful error handling — no stack trace exposure	0 stack-trace exposures	**High**
Scalability	Logical separation of core business domains	Modular boundary enforcement	**Medium**
Scalability	Asynchronous execution of non-critical operations	Non-blocking main thread	**Medium**
Maintainability	Audit logging of critical state-changing operations	100% critical ops logged	**Medium**

7. 3.1 Performance & Efficiency

The following requirements define the minimum acceptable performance characteristics of the system under normal and peak operating conditions.

NFR-PER-01: The system shall calculate and render a fully optimised group itinerary, comprising up to twenty (20) scheduled activities, and present the result to the requesting client within three (3.0) seconds of the initiation of the optimisation request.

NFR-PER-02: Under a concurrent load of up to one hundred (100) simultaneously authenticated users, the system shall process and respond to all standard HTTP requests within five hundred (500) milliseconds, measured from the moment the server receives the complete request to the moment the first byte of the response is transmitted.

NFR-PER-03: The system shall retrieve frequently accessed, short-lived data — including, but not limited to, user session states and draft activity records — from an in-memory caching layer within fifty (50) milliseconds, thereby minimising unnecessary read operations on the primary relational database.

8. 3.2 Security & Privacy

The following requirements define the mandatory security controls that the system must enforce to protect user data, prevent unauthorised access, and ensure regulatory alignment.

8.1 NFR-SEC-01 — Data Encryption at Rest

NFR-SEC-01: All highly sensitive user documents stored by the system — specifically including, but not limited to, passport scans and national identity documents — shall be encrypted at rest using an industry-standard, symmetric block-cipher algorithm equivalent to or stronger than AES-256. No plaintext copy of such documents shall persist on any storage medium managed by the system.

8.2 NFR-SEC-02 — Token-Based Authentication

NFR-SEC-02: All API endpoints that expose protected resources shall require a valid, cryptographically signed bearer token for access. Short-lived access tokens shall expire no later than one (1) hour following issuance. Long-lived refresh tokens, used solely to obtain new access tokens, shall expire no later than twenty-four (24) hours following issuance.

8.3 NFR-SEC-03 — Role-Based Access Control

NFR-SEC-03: The system shall enforce Role-Based Access Control (RBAC) on all write-operations targeting financial data and settlement workflows. A request to perform such an operation shall be rejected with an appropriate HTTP 403 Forbidden response unless the authenticated principal possesses a delegated authoritative role (e.g., Trip Organiser or delegated Financial Editor) as recorded in the system's permission registry.

8.4 NFR-SEC-04 — Rate Limiting

NFR-SEC-04: The system shall implement per-user rate limiting on all publicly accessible endpoints. No single authenticated or unauthenticated user identity shall be permitted to exceed one

hundred (100) requests per minute. Requests that exceed this threshold shall receive an HTTP 429 Too Many Requests response, preventing resource exhaustion and denial-of-service abuse.

9. 3.3 Reliability & Availability

The following requirements define the system's expected behaviour with respect to data availability, fault tolerance, and recovery under adverse network or operational conditions.

9.1 NFR-REL-01 — Offline Resilience

NFR-REL-01: The mobile and web client application shall maintain a local persistence layer capable of storing the complete state of all confirmed itinerary items. In the total absence of network connectivity (0% availability), an authenticated user shall be able to view all previously synchronised, confirmed itinerary content without degradation of the presented data.

9.2 NFR-REL-02 — State Synchronisation

NFR-REL-02: Upon the restoration of network connectivity following a period of offline operation, the system shall complete the full synchronisation of locally accumulated changes to the authoritative server-side state within five (5) seconds. Conflicts arising from concurrent edits shall be resolved deterministically by applying a Server-is-Truth resolution strategy, whereby the server-side version is treated as canonical and local divergences are resolved accordingly.

9.3 NFR-REL-03 — Automated Data Backups

NFR-REL-03: The system shall perform automated, full backups of all persistent data stores on a daily schedule, without requiring manual intervention. Each backup artefact shall be retained in secure storage for a minimum period of seven (7) days, after which it may be purged. The backup mechanism shall be verifiable and shall not impact system availability during execution.

10. 3.4 Usability & User Experience

The following requirements define the interaction efficiency and error communication standards that the system's user interface must satisfy.

10.1 NFR-USA-01 — Interaction Efficiency

NFR-USA-01: The user interface shall be designed such that a Trip Member with basic digital literacy is able to complete any core, repetitive task — including, but not limited to, submitting an activity RSVP or recording a shared expense — within a maximum of five (5) discrete user interactions (clicks or taps) from the relevant entry point. No such task shall require fewer than three (3) interactions in order to prevent accidental submissions.

10.2 NFR-USA-02 — Graceful Error Communication

NFR-USA-02: In the event of any network failure, server-side processing error, or validation rejection, the system shall present the end-user with a clear, localised, and human-readable error message appropriate to the context of the failure. The system shall, under no circumstances, expose raw technical information — including stack traces, internal exception messages, database query errors, or server-side file paths — to the end-user interface.

11. 3.5 Scalability & Maintainability

The following requirements define the structural and operational quality attributes that ensure the system remains adaptable, auditable, and performant as scope and user-base grow.

11.1 NFR-SCA-01 — Modular Domain Separation

NFR-SCA-01: The system architecture shall enforce strict logical separation between each core business domain — including, but not limited to, User Management, Itinerary & Logistics, Financial Settlement, Collaboration, and Document & Safety Management. Inter-domain dependencies shall be expressed through well-defined interfaces or service boundaries, such that the modification or future extraction of any single domain does not necessitate changes to unrelated domains.

11.2 NFR-SCA-02 — Asynchronous Non-Critical Operations

NFR-SCA-02: All system operations that are characterised by high latency or that are non-critical to the immediate completion of the originating user request — including, but not limited to, generating automated daily briefing notifications and dispatching outbound email or push alerts — shall be executed asynchronously. Such operations must not block the primary synchronous request-response lifecycle or delay the delivery of the HTTP response to the client.

11.3 NFR-SCA-03 — Audit Logging

NFR-SCA-03: The system shall automatically generate a structured, tamper-evident audit log entry for every critical state-changing operation performed within the system. Such operations include, but are not limited to, financial settlement submissions, expense modifications, role-delegation changes, and document access events. Each log entry shall record, at minimum: a precise UTC timestamp, the unique identifier of the authenticated principal performing the action, the type of operation performed, and the identity of the affected resource.

5 .Other Requirements

The system must comply with applicable data protection principles given that it stores personally identifiable information including names, email addresses, passport scans, and emergency contacts. Users must be informed of what data is stored and must consent at registration. All stored passwords must be hashed using a modern algorithm (bcrypt or Argon2); plaintext passwords must never be persisted. The

system should provide a mechanism for a user to delete their account and associated personal data.

6 .Initial Results

The initial results of the project include a complete Software Requirements Specification (SRS), including functional and non-functional requirements.

The system has been analyzed and modeled using use cases and diagrams, which provide a clear understanding of system behavior and structure.

7 Revised Project Plan

The project plan has been updated to include the next phases of development:

- Design phase (UML diagrams, architecture)
- Implementation phase (coding using Laravel and PHP)
- Testing phase

The team will continue to collaborate and divide tasks to ensure timely completion.

8 References

- https://www.wanderlog.com/
- https://www.tripit.com/

12. Appendix A: Glossary

- Trip Leader: A Member with elevated permissions who created or manages a trip
- Member: A authenticated user who has been invited to participate in a trip
- Activity: A scheduled event within a trip's itinerary
- Settlement: The resolution of outstanding shared expenses between trip members
- Admin: System manager

Appendix B: IV & V Report

(Independent verification & validation)

IV & V Resource

Name Signature

S#	Defect Description	Origin Stage	Status	Fix Time	
				Hours	Minutes
1					
2					
3					
…					

Table 9: List of non-trivial defects